OS: Practical-1

Roll No: 25MCA009

1. Who:

It shows **who is logged into the system** right now. shows details like username, terminal, login time, and sometimes the remote host.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~$ who
aryan_chaudhari pts/1        2025-09-02 03:04
```

2. Whoami:

It shows **the name of the user who is currently logged in**.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~$ whoami
aryan_chaudhari
```

3. Touch:

Touch command's core function is to modify and access timestamps of an existing file or directories, yet it is used to even create new and empty files.

```
/home/aryan_chaudhari/.hushlogin file.
aryan_chaudhari@LAPTOP-MCK9RUN6:~$ ls
Aryan_Chaudhari
aryan_chaudhari@LAPTOP-MCK9RUN6:~$ cd Aryan_Chaudhari
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls
Aryan.txt           b.txt      clac.sh   file1.txt  lab3.sh   practicle.sh   report1.txt  table.sh
Practicle3.sh       bunny.txt  dest.txt  file2.txt  list.txt  practicle5.sh  report2.txt  third_file.txt
Practicle3.sh.save  calc.sh    emp.txt   file3.txt  mul5.sh   report.txt     shuf.sh
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ touch aaaaaaaaaaaaa.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls
Aryan.txt           aaaaaaaaaaaaa.txt  calc.sh   emp.txt    file3.txt  mul5.sh        report.txt   shuf.sh
Practicle3.sh       b.txt              clac.sh   file1.txt  lab3.sh    practicle.sh   report1.txt  table.sh
Practicle3.sh.save  bunny.txt          dest.txt  file2.txt  list.txt   practicle5.sh  report2.txt  third_file.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
```

4. Cat: Stands for **"concatenate"**.

It is used to **view, combine, and create files**.

Cat file.txt – show file content

Cat file1.txt file2.txt – show multiple files together

Cat > newfile.txt – create a new file

Cat >> file.txt – append content to existing file

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cat file1.txt
Hey, I am Aryan Chaudhari
I'm from gandhinagar
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cat >> file1.txt
HELLO WORLD!!^C
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cat file1.txt
Hey, I am Aryan Chaudhari
I'm from gandhinagar
HELLO WORLD!!aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cat
```

5. Cp:

Stands for **copy** → used to copy files or directories. while using options like -r (recursive) allows copying entire directories along with their contents.

cp -r folder1 folder2: Copy a folder (with everything inside)

cp -i file1.txt file2.txt: Copy with interactive mode (asks before overwrite)

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cat file1.txt
Hey, I am Aryan Chaudhari
I'm from gandhinagar
HELLO WORLD!!aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cat copyfile.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cp file1.txt copyfile.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cat copyfile.txt
Hey, I am Aryan Chaudhari
I'm from gandhinagar
HELLO WORLD!!aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
```

6. Rm:

Stands for **remove** → used to **delete files or directories**.

Be careful → once deleted, it usually can't be recovered!

rm -r foldername: Remove a directory (with everything inside)

rm -i file1.txt: Remove but ask before deleting

rm -f file1.txt: Force remove

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls
copyfile.txt  file1.txt  file2.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ rm file2.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls
copyfile.txt  file1.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
```

7. Mv:

Stands for **move** → used to **move or rename** files/directories.

mv file.txt /home/aryan/Documents/: Move a file to another folder

mv oldname.txt newname.txt: Rename a file

mv folder1 folder2:

If folder2 exists → folder1 will be moved inside it.

If folder2 does NOT exist → folder1 will be renamed to folder2.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls
file1.txt  file2.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ mv file2.txt file1.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls
file1.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
```

8. Ln:

It is used to **create links between files**. By default, it makes **hard links**, while using the -s option creates **symbolic (soft) links**, which act like shortcuts pointing to the original file.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ln file1.txt link1.txt
```

9. Ls:

It **lists files and folders** in the current directory (where you are). By adding options (like -l, -a, -h), it can show detailed information such as permissions, hidden files, file sizes, and modification dates.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls -l
total 0
-rw-r--r-- 2 aryan_chaudhari aryan_chaudhari 0 Sep  2 03:18 file1.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 0 Sep  2 03:18 file2.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 0 Sep  2 03:19 link.txt
-rw-r--r-- 2 aryan_chaudhari aryan_chaudhari 0 Sep  2 03:18 link1.txt
```

10.Chmod: Stands for **change mode** → used to **change permissions** of a file or folder.

Each file/folder has 3 types of users:

- **u** → user (owner)
- **g** → group (team)

- **o** → others (everyone else)
- **a** → all (u+g+o)

And 3 types of permissions:

- **r** → read (4)
- **w** → write (2)
- **x** → execute (1)

chmod 644 file.txt → owner read/write, others read only.

chmod 755 script.sh → owner full, others read/execute.

chmod 777 folder → everyone full (dangerous).

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls -l
total 32
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari  76 Sep  6 05:52 Aryan.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:37 b.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 135 Sep  2 11:45 dest.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 215 Sep  2 12:40 file1.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:40 file2.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:08 file3.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 701 Sep  2 12:37 lab3.sh
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 269 Sep  2 12:40 third_file.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ chmod 777 Aryan.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls -l
total 32
-rwxrwxrwx 1 aryan_chaudhari aryan_chaudhari  76 Sep  6 05:52 Aryan.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:37 b.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 135 Sep  2 11:45 dest.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 215 Sep  2 12:40 file1.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:40 file2.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:08 file3.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 701 Sep  2 12:37 lab3.sh
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 269 Sep  2 12:40 third_file.txt
```

11.Umask:

It controls the default permissions for newly created files and directories.

How it Works:

1. **Linux starts with maximum default permissions:**
    - Files → 666 (rw-rw-rw-) → everyone can read & write
    - Directories → 777 (rwxrwxrwx) → everyone can read, write, enter
2. **Then umask subtracts permissions.**

- Example: umask 022
  - Files: 666 - 022 = 644 → rw-r--r--
  - Dirs: 777 - 022 = 755 → rwxr-xr-x

So umask decides what's not allowed by default.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ umask 0006
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ umask
0006
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ touch bunny.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls -l
total 32
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari  76 Sep  6 05:52 Aryan.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:37 b.txt
-rw-rw---- 1 aryan_chaudhari aryan_chaudhari   0 Sep  6 07:29 bunny.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 135 Sep  2 11:45 dest.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 215 Sep  2 12:40 file1.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:40 file2.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:08 file3.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 701 Sep  2 12:37 lab3.sh
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 269 Sep  2 12:40 third_file.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
```

12.Pwd:

Stands for **print working directory**.
It shows the **full path** of the folder where you are currently standing

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~$ pwd
/home/aryan_chaudhari
```

13.Mkdir:

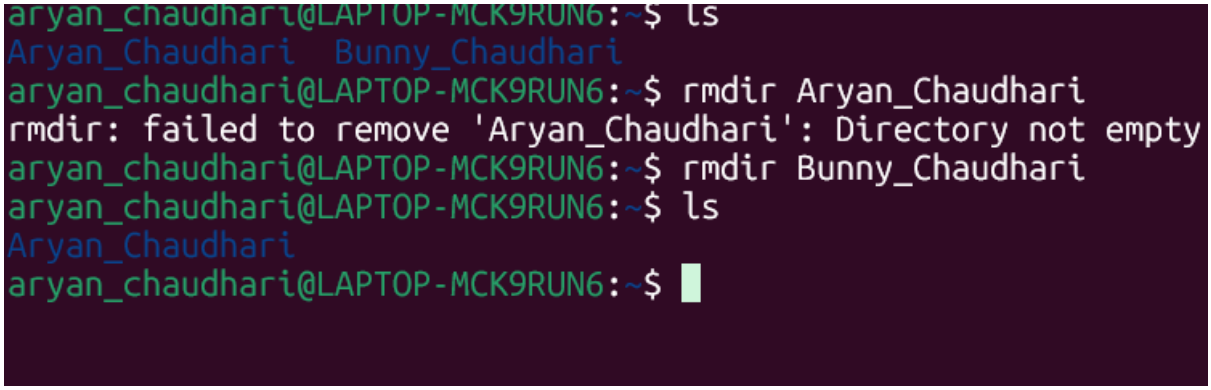
Stands for **make directory** → used to **create new folders**.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cd ..
aryan_chaudhari@LAPTOP-MCK9RUN6:~$ ls
Aryan_Chaudhari
aryan_chaudhari@LAPTOP-MCK9RUN6:~$ mkdir Bunny_Chaudhari
aryan_chaudhari@LAPTOP-MCK9RUN6:~$ ls
Aryan_Chaudhari  Bunny_Chaudhari
aryan_chaudhari@LAPTOP-MCK9RUN6:~$
```

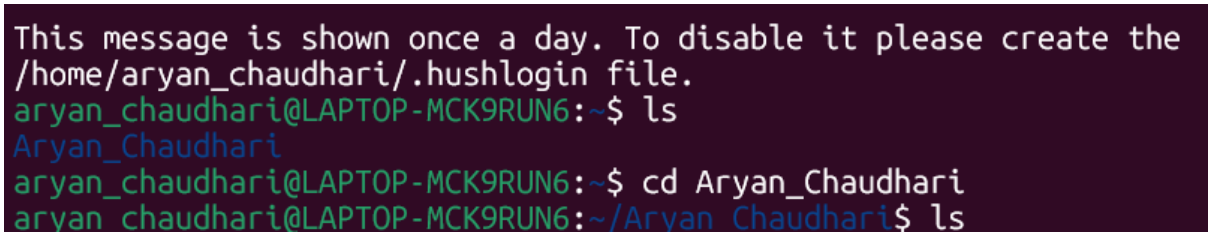
14. Rmdir:

Stands for **remove directory** → used to **delete empty folders** only.

For removing non-empty directories, rm -r is used instead.

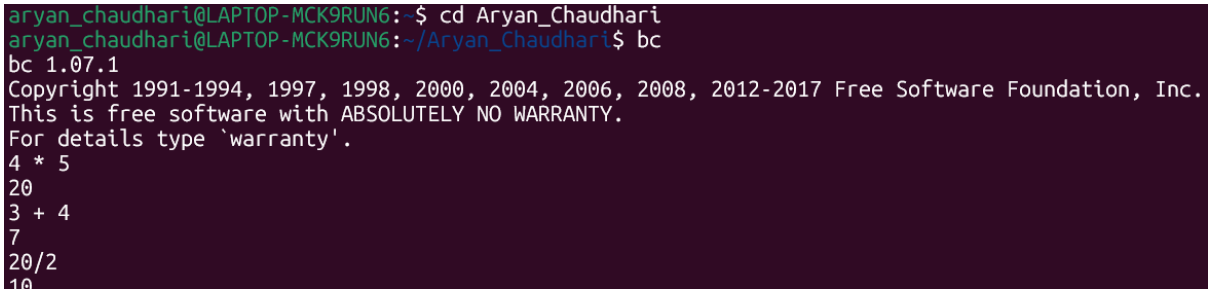


15. Cd: cd stands for change directory, used for the same.



16. Bc: Stands for Basic Calculator.

It's a command-line calculator in Linux that supports normal arithmetic and even advanced math.



17. Expr:

It is used to **evaluate expressions**. It can perform arithmetic operations, string comparisons, find string length, extract substrings, and more, making it useful in shell scripting for simple calculations and text processing.

Expr length “hello”

Expr substr “linux” 2 3(2 nd position, 3 char)

Expr index “abcder” “d”

```
n502@n502:~$ expr 2 + 3
5
```

18.Factor: It is used to **display the prime factors of a given number.**

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ factor 400
400: 2 2 2 2 5 5
```

19.Logname:

It prints the **name of the user who originally logged in** to the current session.

```
n502@n502:~$ logname
n502
```

20.Uname:

Uname command is used to print the system info, without flag it provides the name of the operating system.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ uname
Linux
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
```

21.tty:

Stands for **Teletypewriter** (old terminals were called teletypes).
It shows the **terminal device file** you are currently using.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ tty
/dev/pts/0
```

22.Date: Shows or sets the **system’s current date and time**.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ date
Tue Sep  2 04:52:51 UTC 2025
```

23.Df:

Stands for **disk filesystem**.
It shows the **disk space usage** of your mounted filesystems.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ df
Filesystem      1K-blocks      Used  Available Use% Mounted on
none              3980380         0    3980380   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none              3980380         4    3980376   1% /mnt/wsl
drivers         251656516 115695440  135961076  46% /usr/lib/wsl/drivers
/dev/sdd       1055762868   1602052 1000457344   1% /
none              3980380       108    3980272   1% /mnt/wslg
none              3980380         0    3980380   0% /usr/lib/wsl/lib
rootfs            3975368      2664    3972704   1% /init
none              3980380       548    3979832   1% /run
none              3980380         0    3980380   0% /run/lock
none              3980380         0    3980380   0% /run/shm
none              3980380        76    3980304   1% /mnt/wslg/versions.txt
none              3980380        76    3980304   1% /mnt/wslg/doc
C:\             251656516 115695440  135961076  46% /mnt/c
D:\             245758972    707036  245051936   1% /mnt/d
tmpfs             3980380        16    3980364   1% /run/user/1000
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
```

24. Du:

Du command is used to estimate the file space usage.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cd ..
aryan_chaudhari@LAPTOP-MCK9RUN6:~$ du
96      ./Aryan_Chaudhari
4       ./.local/share/nano
8       ./.local/share
12      ./.local
4       ./.landscape
4       ./.cache
140     .
```

25. Ulimit: Stands for **“user limit”**.

It shows or controls the **resource limits** that the shell and its child processes can use.
(Like how much memory, CPU, file size, open files etc. a program is allowed to use).

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ulimit
unlimited
```

26. Cal: call is used to display the calendar.

```
┌──(jarvis㉿Jarvis)-[~]
└─$ cal
   September 2025
Su Mo Tu We Th Fr Sa
    1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30
```

27. Wc: Stands for **"word count"**.

It counts **lines, words, characters, and bytes** in a file (or input).

-l → counts lines.

-w → counts words.

-c → counts bytes.

-m → counts characters

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ wc file1.txt
 3  3 22 file1.txt
```

28.Sort:

It is used to arrange lines of text in a file (or input) in a specific order.
By default → sorts in alphabetical (dictionary) order.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cat > demo.txt
dddd
ccc
bb
a^C
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ sort demo.txt
bb
ccc
dddd
```

29. Cut:

It is used to **extract specific sections of text from each line of a file or input**.

-c → Select Character

Cut -c1-5 file.txt

Prints first 5 character of each line

-d → Change delimiter

Cut -d "," -f1 file1.csv

Print the first column from a CSV file.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ cut -d ' ' -f2 aryan.txt
World!!
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
```

30.Grep:

It is used to **search for specific patterns of text** within files or input. It supports regular expressions and can highlight matching lines, making it very powerful for filtering logs, code, or large text files.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ grep 'Hello' aryan.txt
Hello World!!
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
```

31. Awk:

AWK is a **command-line tool** and also a **tiny programming language**.

It works on **text files** or **command output**, line by line.

Ex: 101 John 78

$0 → whole line → 101 John 78

$1 → first field → 101

$2 → second field → John

$3 → third field → 78

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ more bunny.txt
X Y Z
A B C
P Q R
1 2 3
8 9 0
E R Y
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ awk '{print $1}' bunny.txt
X
A
P
1
8
E
```

32.Head:

It is used to **display the beginning of a file**. By default, it shows the first 10 lines, but with the -n option, you can specify any number of lines to view.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ more Aryan.txt
Hello World
Hello Banana
Hello Apple
hiii hiii hii
a
b
c
d
e
f
g
h
i
j
k
l
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ head -n 5 Aryan.txt
Hello World
Hello Banana
Hello Apple
hiii hiii hii
a
```

33.Tail:

It is used to **display the end of a file.** By default, it shows the last 10 lines, but with the -n option you can choose how many lines to display. Using tail -f lets you **monitor a file in real-time,** which is especially useful for log files.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ more Aryan.txt
Hello World
Hello Banana
Hello Apple
hiii hiii hii
a
b
c
d
e
f
g
h
i
j
k
l
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ tail Aryan.txt
c
d
e
f
g
h
i
j
k
l
```

34. More:

The more command in Linux is a pager utility – it lets you view the contents of a file one screen at a time (instead of dumping the whole file at **once like cat).**

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls
b.txt  dest.txt  file1.txt  file2.txt  file3.txt  lab3.sh  third_file.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ more file1.txt
101, Aryan, MCA, 99 \n
102, Niraj, MCA, 78
103, Shyam, MCA, 95 \n
104, Subh, MCA, 23
105, Harsh, MCA, 34
106, Bunny, MCA, 45
107, Vanshika, MCA, 66 \n
108, Virangi, MCA, 33
109, Piyu, MCA, 35
110, Kenil, MCA, 77 \n
```

35.Tee:

Whatever output comes from a command:

1. It displays it on the screen (terminal).
2. It writes (saves) it into a file at the same time.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls -l | tee list.txt
total 36
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari  76 Sep  6 05:52 Aryan.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:37 b.txt
-rw-rw---- 1 aryan_chaudhari aryan_chaudhari  37 Sep  6 08:03 bunny.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 135 Sep  2 11:45 dest.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 215 Sep  2 12:40 file1.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:40 file2.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:08 file3.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 701 Sep  2 12:37 lab3.sh
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 269 Sep  2 12:40 third_file.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls  -l
total 40
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari  76 Sep  6 05:52 Aryan.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:37 b.txt
-rw-rw---- 1 aryan_chaudhari aryan_chaudhari  37 Sep  6 08:03 bunny.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 135 Sep  2 11:45 dest.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 215 Sep  2 12:40 file1.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:40 file2.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 203 Sep  2 12:08 file3.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 701 Sep  2 12:37 lab3.sh
-rw-rw---- 1 aryan_chaudhari aryan_chaudhari 655 Sep  6 08:25 list.txt
-rw-r--r-- 1 aryan_chaudhari aryan_chaudhari 269 Sep  2 12:40 third_file.txt
```

36. Ps:

The ps command in Linux stands for Process Status.
It is used to list the processes currently running on the system, along with details like PID, user, CPU usage, memory usage, etc.

-e OR -A → Show all processes

-ef → Full list of all processed in system

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ps -a
    PID TTY          TIME CMD
    385 pts/1    00:00:00 bash
   1455 pts/0    00:00:00 ps
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 05:48 ?        00:00:00 /sbin/init
root           2       1  0 05:48 ?        00:00:00 /init
root           7       2  0 05:48 ?        00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root          40       1  0 05:48 ?        00:00:00 /usr/lib/systemd/systemd-journald
root          89       1  0 05:48 ?        00:00:00 /usr/lib/systemd/systemd-udevd
systemd+     113       1  0 05:48 ?        00:00:00 /usr/lib/systemd/systemd-resolved
systemd+     117       1  0 05:48 ?        00:00:00 /usr/lib/systemd/systemd-timesyncd
root         174       1  0 05:48 ?        00:00:00 /usr/sbin/cron -f -P
message+     175       1  0 05:48 ?        00:00:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
root         188       1  0 05:48 ?        00:00:00 /usr/lib/systemd/systemd-logind
root         190       1  0 05:48 ?        00:00:00 /usr/libexec/wsl-pro-service -vv
root         195       1  0 05:48 hvc0     00:00:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
syslog       201       1  0 05:48 ?        00:00:00 /usr/sbin/rsyslogd -n -iNONE
root         209       1  0 05:48 tty1     00:00:00 /sbin/agetty -o -p -- \u --noclear - linux
root         220       1  0 05:48 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
root         249       2  0 05:48 ?        00:00:00 /init
root         251     249  0 05:48 ?        00:00:00 /init
aryan_c+     254     251  0 05:48 pts/0    00:00:00 -bash
root         255       2  0 05:48 pts/1    00:00:00 /bin/login -f
aryan_c+     371       1  0 05:48 ?        00:00:00 /usr/lib/systemd/systemd --user
aryan_c+     373     371  0 05:48 ?        00:00:00 (sd-pam)
aryan_c+     385     255  0 05:48 pts/1    00:00:00 -bash
polkitd      767       1  0 05:55 ?        00:00:00 /usr/lib/polkit-1/polkitd --no-debug
aryan_c+    1456     254  0 06:11 pts/0    00:00:00 ps -ef
```

37.Kill: kill command is used to send a signal to process.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~$ cd Aryan_Chaudhari
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

38. Nice:

In Linux, nice is used to set the priority of a process when you start it.

Linux uses a niceness value (nice value) between -20 to +19.

- -20 = highest priority (very fast).
- 0 = default priority.
- +19 = lowest priority (very slow, “be nice to others”).

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ nice --help
Usage: nice [OPTION] [COMMAND [ARG]...]
Run COMMAND with an adjusted niceness, which affects process scheduling.
With no COMMAND, print the current niceness.  Niceness values range from
-20 (most favorable to the process) to 19 (least favorable to the process).

Mandatory arguments to long options are mandatory for short options too.
  -n, --adjustment=N   add integer N to the niceness (default 10)
      --help        display this help and exit
      --version     output version information and exit

NOTE: your shell may have its own version of nice, which usually supersedes
the version described here.  Please refer to your shell's documentation
for details about the options it supports.

Exit status:
  125  if the nice command itself fails
  126  if COMMAND is found but cannot be invoked
  127  if COMMAND cannot be found
  -    the exit status of COMMAND otherwise

GNU coreutils online help: <https://www.gnu.org/software/coreutils/>
Report any translation bugs to <https://translationproject.org/team/>
Full documentation <https://www.gnu.org/software/coreutils/nice>
or available locally via: info '(coreutils) nice invocation'
```

39.Echo:

It is used to **display a line of text or variable values** to the terminal. It is commonly used in shell scripts to print messages, show environment variables, or generate formatted output.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ echo "Hello world"
Hello world
```

40.Read:

It is used to **take input from the user** and store it in a variable. It is often used in shell scripts to interact with users by asking for values like names, passwords, or options during execution.

We use **read -p** to make the script **more user-friendly**.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ read -p "enter Your name:"
enter Your name:aryan
```

41.Top:

top command is like task displayer for linux, it displays all the current processes in runtime.

```
top - 04:12:50 up 20 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  23 total,   1 running,  22 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.5 sy,  0.0 ni, 99.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7774.2 total,   7263.8 free,    509.1 used,    151.6 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7265.0 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   21648  12256   9312 S   0.0   0.2   0:00.74 systemd
      2 root      20   0    3060   1664   1664 S   0.0   0.0   0:00.01 init-systemd(Ub
      7 root      20   0    3060   1792   1792 S   0.0   0.0   0:00.00 init
     43 root      19  -1   50428  15488  14592 S   0.0   0.2   0:00.21 systemd-journal
     92 root      20   0   25136   6272   4864 S   0.0   0.1   0:00.29 systemd-udevd
    105 systemd+  20   0   21456  11776   9856 S   0.0   0.1   0:00.21 systemd-resolve
    106 systemd+  20   0   91024   7680   6784 S   0.0   0.1   0:00.13 systemd-timesyn
    177 root      20   0    4236   2432   2304 S   0.0   0.0   0:00.01 cron
    178 message+  20   0    9632   4864   4480 S   0.0   0.1   0:00.11 dbus-daemon
    191 root      20   0   17968   8320   7424 S   0.0   0.1   0:00.12 systemd-logind
    193 root      20   0 1755840  12032  10240 S   0.0   0.2   0:00.18 wsl-pro-service
    204 root      20   0    3160   1920   1792 S   0.0   0.0   0:00.01 agetty
    212 syslog    20   0  222508   5632   4480 S   0.0   0.1   0:00.11 rsyslogd
    216 root      20   0    3116   1792   1664 S   0.0   0.0   0:00.00 agetty
    222 root      20   0  107032  22272  13056 S   0.0   0.3   0:00.19 unattended-upgr
    318 root      20   0    3064    896    896 S   0.0   0.0   0:00.00 SessionLeader
    319 root      20   0    3080   1024   1024 S   0.0   0.0   0:00.04 Relay(321)
    321 aryan_c+  20   0    6072   5248   3584 S   0.0   0.1   0:00.08 bash
    322 root      20   0    6692   4224   3584 S   0.0   0.1   0:00.01 login
    412 aryan_c+  20   0   20300  11264   9216 S   0.0   0.1   0:00.14 systemd
    413 aryan_c+  20   0   21156   3520   1792 S   0.0   0.0   0:00.00 (sd-pam)
    424 aryan_c+  20   0    6056   4992   3456 S   0.0   0.1   0:00.03 bash
    508 aryan_c+  20   0    9300   5248   3072 R   0.0   0.1   0:00.01 top
```

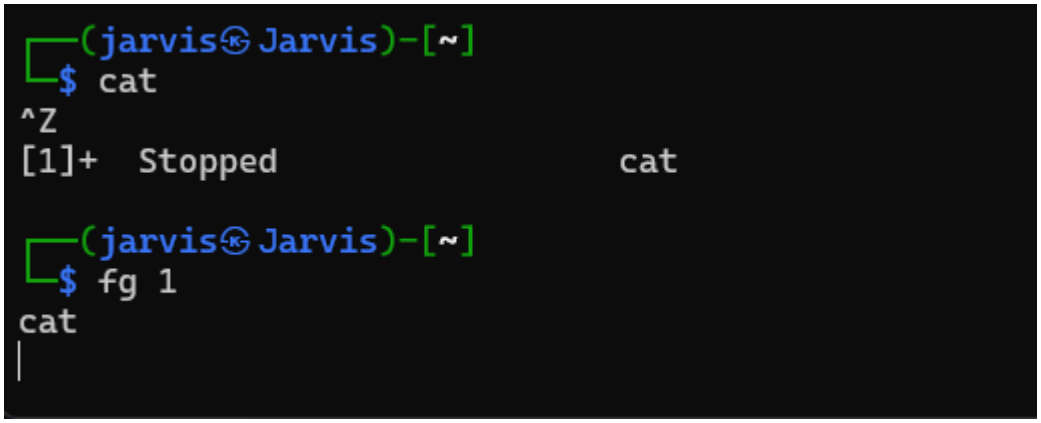
42.Bg:

bg command is used to put an ongoing process to background and in waiting mode.

When used without any arguments it is used to show all the background tasks.

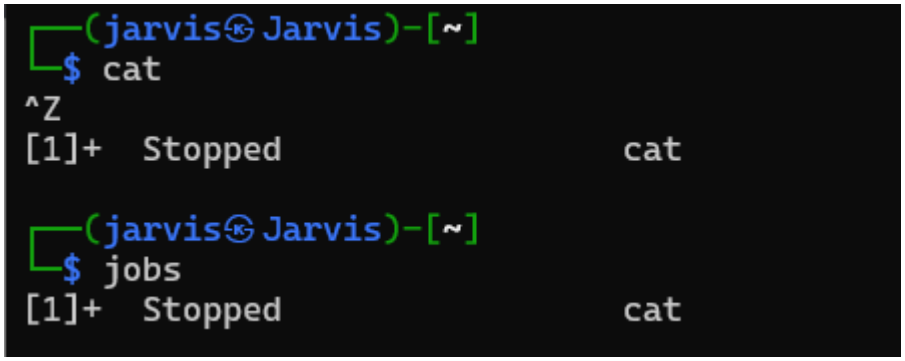
43. Fg:

fg command is used to convert a background job to forehand, making the waiting process start executing again.



44. Jobs:

Jobs command is used to display all current jobs.



45.Pipe

It is used to **pass the output of one command as input to another.** It allows chaining multiple commands together for efficient processing.

```
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls -1
Aryan.txt
b.txt
bunny.txt
dest.txt
file1.txt
file2.txt
file3.txt
lab3.sh
third_file.txt
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$ ls -1 | wc -l
9
aryan_chaudhari@LAPTOP-MCK9RUN6:~/Aryan_Chaudhari$
```